

CMR SPECIALIST · INFERENCE

Gemini Connection: Verification Report

Every test run while connecting the system to Gemini — what each test was trying to prove, how it was done, what happened, and what was concluded. Written for a non-technical reader: each test is explained first in plain words, then in detail.

Item	Detail
Goal	Prove the system can reach Gemini, send correctly formed requests, and degrade safely when Google cannot answer
Method	Offline dry runs (no network) → direct API probes → live end-to-end cases
Result	Wiring proven correct; 4 findings fixed in code; final blocker is Google-side rate limiting (transient), not our system
Test suite	25 automated checks green throughout (mock mode unaffected)

NOTE. Bottom line up front: your API key is valid, the code is correct, and Gemini answered successfully more than once. The only outstanding item is Google's temporary throttling, which resolves with time — see Section 5.

1 · Test log (chronological)

Each row is one test run. PASS means the thing being proven held true; FAIL means it surfaced a problem (which Section 2 then explains).

#	Test (plain words)	Verdict
T1	Offline dry run: with a fake key and no network, does the system build the right request and survive failure gracefully?	PASS
T2	First live case (ownership complaint): does Gemini classify and extract names?	FAIL
T3	Ask Google directly: is the problem our key, our address, or our model name?	PASS
T4	After fixing the model name: does a live case use Gemini without errors?	FAIL
T5	Ask the model a trivial question directly: is Google reachable at all?	PASS
T6	Ask for name extraction directly: what exactly comes back?	FAIL
T7	Same extraction request with a 7x larger token budget: does the answer complete?	PASS
T8	Server restart + live ownership case with fixed budgets: do names land in the DB?	FAIL
T9	Automated suite re-run after adding retry logic: did anything regress?	PASS
T10	Final live case: names in DB?	FAIL

What each test did, in detail

T1 — dry run. Set a fake key, intercepted the HTTP layer so nothing left the machine, and fired a proposal. Verified three things: the provider object points at Google's OpenAI-compatible address with model gemini-2.0-flash; the outgoing payload contains the model name and Bearer authorization; and a simulated network failure returns empty-handed instead of crashing. All three held.

T2 — first live case. Submitted a change-of-ownership complaint through the real API. The case was created and the task queued, but zero LLM-extracted fields landed and the log showed HTTP 404 from Google. A 404 means 'wrong address or wrong name' — the investigation moved to T3.

T3 — direct interrogation. Called Google's endpoint with curl and printed the full error body. Google answered plainly: the key is fine, the address is fine, but gemini-2.0-flash is retired — use gemini-3.8-flash. Diagnosis complete in one call.

T4 — retry after model fix. Updated the model name in settings and .env, restarted both processes. The live case ran with zero LLM errors — but still zero LLM fields. No error plus no result meant the model was answering with something unparseable. Moved to T5/T6.

T5 — trivial probe. Asked the model to reply 'ok'. It did. Conclusion: reachable, authenticated, correct model — the plumbing is fine; the problem is specific to the extraction request.

T6 — raw extraction reply. Printed the model's exact unedited response to the name-extraction prompt. It came back cut off mid-sentence after a handful of tokens. A reply that stops that early, on a tiny request, points at the token budget being consumed before visible output — the signature of a thinking model doing hidden reasoning inside a 150-token allowance.

T7 — budget x7. Identical request with 1024 tokens. Perfect JSON came back: buyer Adaeze Okafor, seller Musa, requester Adaeze Okafor. Hypothesis confirmed; budgets were raised in code (intent/extraction 150 to 600, drafting 300 to 800).

T8 — live case, fixed budgets. Only the regex plate number landed; the log showed one HTTP 503 (service unavailable) from Google. The request was well-formed this time — Google itself was refusing. Response: added one-retry-with-pause resilience to the proposal helper.

T9 — regression suite. 12 passed covering the new retry code and all pre-existing behavior. Nothing regressed; mock mode byte-identical.

T10 — final live case. Still regex-only fields; Google 503 persisted across both the attempt and its retry. Direct probes minutes earlier had succeeded, so configuration is ruled out — this is upstream throttling. All test traffic stopped to let quota recover.

2 · Findings (4)

Finding 1 — retired model name (solved)

Symptom: HTTP 404 on every call. Evidence: Google's error body naming gemini-2.0-flash as retired and prescribing gemini-3.8-flash. Fix: model name updated in settings.py and .env. Lesson: model names are perishable — keep them in one configurable place (done) and re-check them when calls start 404ing.

Finding 2 — thinking model ate the token budget (solved)

Symptom: replies cut off after a few tokens; JSON unparseable. Evidence: identical prompt succeeds at 1024 tokens and returns perfect JSON. Fix: budgets raised (600 intent/extraction, 800 drafting). Lesson: never size token budgets for the visible answer alone — reasoning models spend invisibly first.

Finding 3 — stale process squatting on port 8080 (solved)

Symptom: fresh server produced no logs and health checks timed out. Investigation: an old server process from an earlier session still held the port, so the new instance could never bind. Fix: killed by process id, restarted clean. Lesson: always confirm the old processes are dead (pgrep) before declaring a startup broken.

Finding 4 — Google-side throttling, 503s (open, transient)

Symptom: well-formed, previously succeeding calls now return 503 Service Unavailable, persisting across retries. Evidence for throttling rather than misconfiguration: the same key, address, and model succeeded minutes earlier (T5/T7); failures began only after a burst of diagnostic calls — the classic free-tier rate-limit pattern. Mitigation in place: one retry with pause; deterministic fallback keeps every case working regardless. Resolution: time — quota windows reset; then verify with a single case (Section 5). If 503s persist into tomorrow, escalate to a different flash model or the paid tier.

3 · Code changes made during verification

File	Change	Why (which test)
configs/settings.py	GEMINI_API_URL + GEMINI_MODEL settings; provider option 'gemini'	So the endpoint and (perishable) model name are configurable, not hard-coded
providers/http_provider.py	Sends the model field in the payload	Gemini's endpoint rejects model-less requests (T2/T3)
inference/gateway.py	'gemini' branch + llm_enabled() flag	One-line provider switch; callers can ask if a real model exists
agent/llm.py (new)	propose_json / propose_text: tolerant parsing, never raise, one retry	The propose side of propose-vs-enforce; T8 drove the retry
agent/intent.py	LLM proposes classification; enum + range enforced; rules stay as fallback	Model judgment without surrendering control
agent/extractor.py	LLM may only add names regex missed, marked pending, never overrides	Recall without trusting the model over proven patterns
agent/service.py	LLM drafts answers from cited spans; citation validator still final	Better prose, same evidence discipline
Token budgets	150 to 600 (intent/extract), 300 to 800 (draft)	Finding 2: thinking-model headroom (T6/T7)

NOTE. Nothing in this table changes mock-mode behavior: with INFERENCE_PROVIDER=mock the new code paths are inert, which is why all 25 automated checks stayed green throughout.

4 · How to read the system after this work

Three log lines tell the whole story. In /tmp/cmr_server.log:

```
# silence = Gemini engaged and contributed (verify via extracted fields)
# LLM proposal failed ... 503 ... = Google throttling; system on deterministic floor
# LLM proposal failed ... 401/404 ... = key or model problem; needs human attention
```

And the one database query that proves augmentation worked:

```
sqlite3 data/cmr_cases.db
"SELECT name, raw_value, confidence, validation
FROM extracted_fields WHERE case_id='<CASE_ID>';
# rows with validation='pending' and confidence ~0.55 are Gemini's contributions;
# 'valid' rows near 0.8-0.95 are the regex floor.
```

5 · Current status & next step

Item	State
API key	Installed in .env; authenticated successfully (5xx, never 401)
Endpoint + model	Correct; gemini-3.8-flash confirmed live by direct probe

Item	State
Wiring (propose paths)	Proven by dry run + live successes; 25/25 tests green
Resilience	Retry + deterministic fallback; verified under real failures
Blocker	Google free-tier throttling (503s) — transient, quota-based

Next step, exactly one: wait roughly an hour without test traffic, submit a single ownership complaint mentioning two names, and query its extracted fields. Names present means Gemini is back; still absent with fresh 503s means the throttle window is longer — repeat once more tomorrow before considering a model or tier change.

WARNING. Deliberately not done: no quota-burning retries, no model hopping, no paid-tier upgrade — all premature while the cause is a time-window throttle the system already rides out gracefully.